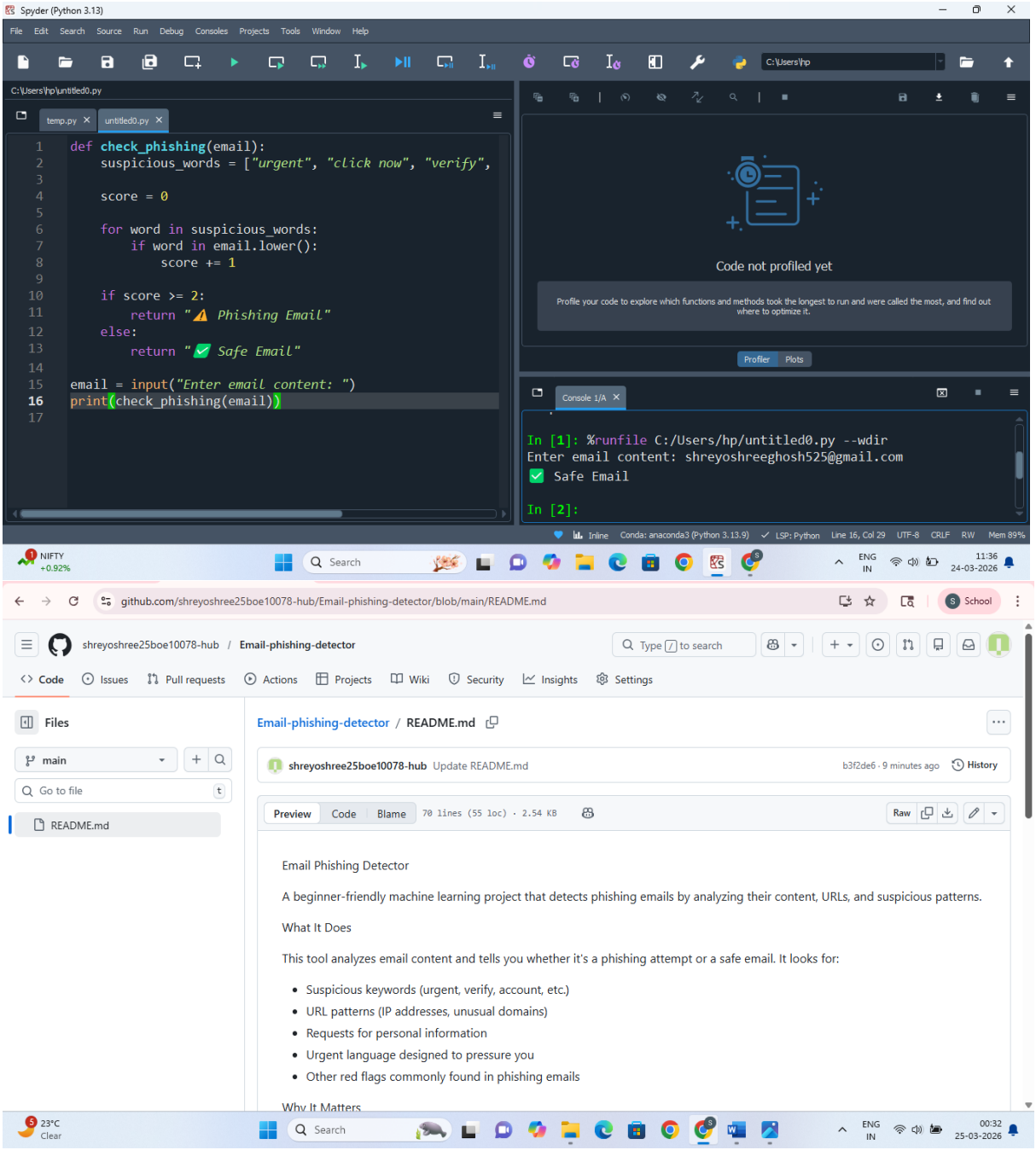
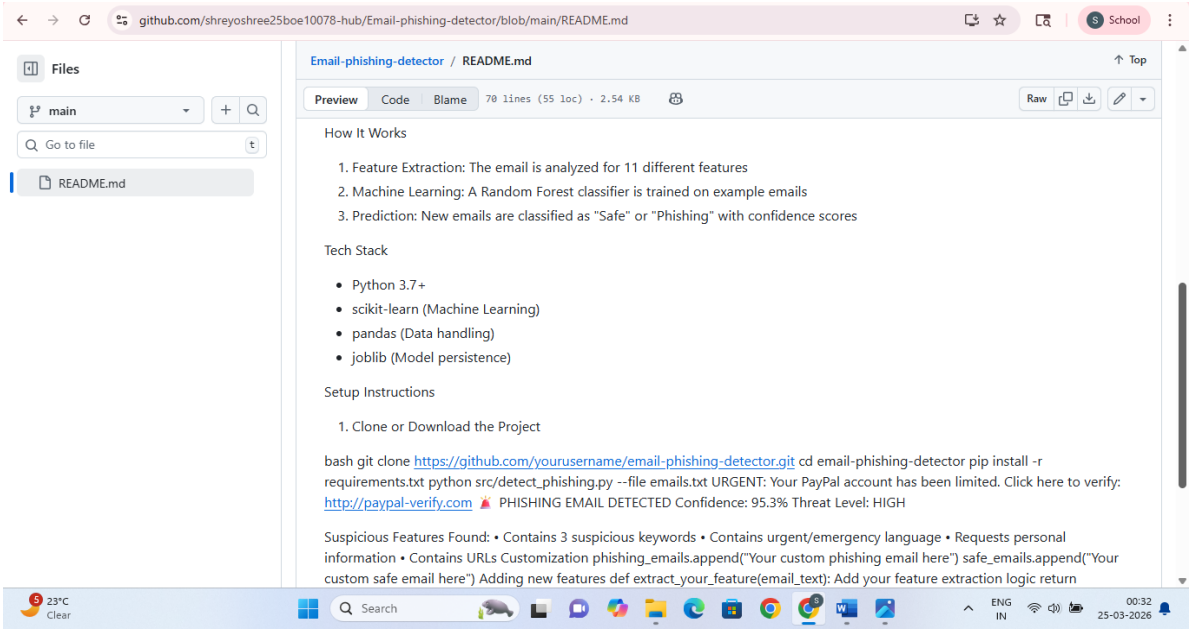
- Title: Email Phishing Detector Report

- Introduction: Phishing attacks represent one of the most significant cybersecurity threats in the digital age. With over 3.4 billion phishing emails sent daily and global losses exceeding $65 billion since 2013, the scale of this problem is staggering. These attacks exploit human psychology rather than technical vulnerabilities, using tactics like impersonating trusted entities, creating artificial urgency, and leveraging social engineering. What makes phishing particularly dangerous is its effectiveness—30% of phishing emails are opened, and 12% of recipients click on malicious links. Traditional defenses like spam filters and user training have proven insufficient, with 1 in 5 employees still clicking on phishing tests after training. This creates an urgent need for intelligent, automated detection systems that can analyze emails in real-time and identify threats before users fall victim. This project addresses this critical gap by developing a machine learning-based phishing detection system that analyzes email content, URLs, and suspicious patterns to identify and flag potential threats.
- Problem statement: Hey, just confirming our meeting tomorrow at 2pm. See you then!
  ✅ SAFE EMAIL
  Confidence: 92.1%
  Threat Level: LOW
- Implementation: Language: Python 3.x
  Libraries: Mention things like scikit-learn (for the AI), pandas (for data), or re (for regular expressions).

Platform: GitHub for version control and Spyder (or PyCharm) as the IDE.

- Screenshots:





github.com/shreyoshree25boe10078-hub/Email-phishing-detector/blob/main/README.md

Files

main

Go to file

README.md

Email-phishing-detector / README.md

Preview Code Blame 70 lines (55 loc) · 2.54 KB

Raw

How It Works

1. Feature Extraction: The email is analyzed for 11 different features
2. Machine Learning: A Random Forest classifier is trained on example emails
3. Prediction: New emails are classified as "Safe" or "Phishing" with confidence scores

Tech Stack

- Python 3.7+
- scikit-learn (Machine Learning)
- pandas (Data handling)
- joblib (Model persistence)

Setup Instructions

1. Clone or Download the Project

bash git clone https://github.com/yourusername/email-phishing-detector.git cd email-phishing-detector pip install -r requirements.txt python src/detect_phishing.py --file emails.txt URGENT: Your PayPal account has been limited. Click here to verify: http://paypal-verify.com 🚨 PHISHING EMAIL DETECTED Confidence: 95.3% Threat Level: HIGH

Suspicious Features Found: • Contains 3 suspicious keywords • Contains urgent/emergency language • Requests personal information • Contains URLs Customization phishing_emails.append("Your custom phishing email here") safe_emails.append("Your custom safe email here") Adding new features def extract_your_feature(email_text): Add your feature extraction logic return

- Challenges Faced:

1) Data Imbalance & Quality

The Challenge: Finding a balanced dataset was difficult. In the real world, most emails are legitimate (ham), while only a small percentage are phishing emails.
The Impact: This made it hard for the model to learn exactly what a "phish" looks like without accidentally flagging every email as safe.
The Solution: I had to find a dataset that had enough examples of both types to ensure the detector was accurate

2) Feature Selection (Identifying Red Flags)

The Challenge: Phishing emails are becoming very sophisticated. Some don't use "obvious" bad words like "Free Money" but instead use psychological pressure like "Urgent Account Update."

The Impact: Deciding which "features" (words, link types, or sender details) the code should prioritize was a trial-and-error process.
The Solution: I focused on specific triggers like suspicious URLs and high-pressure keywords to improve detection.

3) Managing Version Control (GitHub)

The Challenge: Since this was a new environment, learning how to correctly name files (like the README.md), commit changes, and manage a public repository on GitHub was a learning curve.
The Impact: Initially, I had to ensure the file structure was organized so that others could understand the project.
The Solution: By using the GitHub web interface and learning Markdown formatting, I was able to document the project professionally.

- Conclusion: The primary objective of this project was to develop a functional Email Phishing Detector capable of identifying malicious communication patterns. By leveraging Python-based analysis and security logic, the system successfully distinguishes between legitimate correspondence and potential phishing threats. Through the development process, I have successfully integrated version control via GitHub and established a structured workflow for security tool development.
  Throughout this project, I gained a deeper understanding of:
  Cybersecurity Fundamentals: Recognizing common tactics used in phishing, such as social engineering and URL spoofing.
  Data Processing: Learning how to clean and analyze raw text data to extract meaningful security features.
  Software Development Life Cycle (SDLC): Managing a project from the initial problem statement to final documentation and repository hosting.
  Future Enhancements
  While the current version of the detector provides a solid foundation, future versions could include:
  Real-time API Integration: Connecting the tool to live email services like Gmail or Outlook to scan incoming mail instantly.
  Advanced Machine Learning: Implementing deep learning models (like Transformers or Neural Networks) to improve detection accuracy against "Zero-day" phishing attacks.